

Deep Learning Assignment 2

INFOMDLR — June 16, 2026 – Team 21

1st Bas de Blok
std. nr. 0934348

2nd Nicolae-Radu Olariu
std. nr. 0545341

3rd Charlie Choffat
std. nr. 9655301

4th Joris Heemskerk
std. nr. 5201284

Abstract—The classification of brain states from neuroimaging data is a common problem in neuroscience and clinical research. In this article, we used machine learning to classify different cognitive states from a small set of magnetoencephalography (MEG) recordings. Namely: resting, motor, working memory, and story & maths. We trained 5 models (a baseline, EEGNet, EEGNet-SA, MEG-GPT, and MEG-GCNet), both on a single subject and a cross-subject partition. Multiple models achieved 100% accuracy on the single subject. EEGNet performed best on the cross-subject partition with an accuracy of 64.20%, showing that generalising over multiple subjects remains challenging.

Index Terms—MEG, Task Classification, Machine learning

I. INTRODUCTION

There are several ways neuroscientists study the human brain. One of the most commonly used techniques is electroencephalogram (EEG), a non-invasive neuroimaging method that measures electric activity (voltage changes) on the scalp, using electrodes [1]. These scans allow neuroscientists to study the working human brain. Deep learning techniques are often applied to EEG data for clinical classification [2]–[4]. Later, a new neuroimaging method was introduced: Magnetoencephalography (MEG) [5]. MEG is also a non-invasive, but unlike EEG, it measures magnetic fields produced by electric currents flowing between and through neurons. This technique passes through the skull and therefore has less distortion, allowing for superior spatial resolution. MEG data has traditionally been studied with advanced signal processing, but more recently it is being studied using AI [6]. The most popular AI task in this domain is brain state classification. Our study aims to expand upon this newly growing body of research by answering the following research question:

To what capacity are modern AI models able to capture patterns in MEG data in order to achieve high classification accuracy under highly data constrained conditions, both within and across multiple subjects.

To answer this question, we trained five, progressively more complex, AI models to a small MEG dataset, consisting of three training and four testing subjects. We first optimised, trained, and tested our models on a single subject to isolate the capacity of each model to fit to the MEG data. Then, we optimised and trained our models on the three training subjects and tested on the four separate testing subjects to evaluate how well the models were able to generalise across subjects.

In this paper, we first introduce our data and classification tasks in Section II. In Section III, we describe our models and

architectural choices. Afterwards, in Section IV, we describe our approach to data preparation, hyperparameter optimising, cross-validation, validation metrics, testing approach, and our hardware restrictions. Section V shows the optimisation results and the train and test performance. In Section VI, we discuss our results.

II. DATA

In this study, we use a dataset provided by Utrecht University. It consists of two components: an intra-subject and a cross-subject partition. The intra-subject partition consists of MEG data recorded from a single subject, while the cross-subject partition contains 6 (2 for training and 4 for testing). Since the intra-subject subject is unique, their training data was added to the cross-subject partition, making it 3 training subjects and bringing the total up to 7. The training data covers 4 classes, corresponding to 4 tasks:

- **rest**: The subject was in a relaxed state.
- **motor**: The subject performs a motor task (e.g., moving fingers or feet).
- **story & maths**: The subject performs a language processing and mental calculation task.
- **Working memory**: The subject performs a memorisation task.

For each task, the training data for both partitions consists of 8 files per subject. The intra-subject partition contains 2 testing files per task, while the cross-subject partition has 4 testing files per task for two of the subjects, and 2 testing files per task for the other two subjects. There is no overlap in subjects for the train and test data of the cross-subject dataset. Each file consists of 35,624 ordered measurements for each of the 248 sensors. Each value is recorded in femtoTesla (fT). The measurements were recorded using a sample rate of 2034 Hz, meaning there is $\frac{1}{2034}$ seconds between each measurement. An illustration of one data file can be found in Appendix A. Unfortunately, we have no access to how the sensors are spatially mapped onto the subjects' heads. This means our data does not contain any clear spatial relations between the sensors. This will influence our model decisions.

III. MODELS

This section describes the different models used and the architectural changes made. The models become progressively more complex and each approach the structure of the dataset in a different manner.

A. Baseline

Firstly, in order to later compare our model performance, we created a simple baseline model. For this, we used a minimalistic Neural Network, with a single hidden layer of 64 neurons, a ReLU activation, and 4 output neurons, corresponding to the 4 classes. We chose 64 neurons, as the input sizes are rather large ($248 \times \text{window_size}$), meaning more neurons could better capture the signal and result in a better baseline model. However, too many neurons makes it too easy for the model to overfit on the data. Preliminary experimentation showed 64 neurons to perform best on the validation data.

B. EEGNet

EEGNet [7] is a compact convolutional neural network specifically designed for EEG- and MEG-based brain-computer interface (BCI) tasks and has since become one of the most widely adopted baselines in neuroimaging deep learning literature, due to its strong performance across a broad range of BCI paradigms. We selected EEGNet as our primary convolutional approach precisely because of this track record: it proves a well-validated reference point against which more complex architectures can be meaningfully compared. The model introduces a strong inductive bias by assuming that sequentially ordered sensors are locally related (i.e., sensors that are close by index should be semantically similar). It realises this through a combination of temporal convolutions, which extract frequency-specific patterns over time, and depth-wise spatial convolutions, which learn spatial filters across locally grouped sensors. This makes it a highly parameter-efficient approach to neuroimaging classification.

C. EEGNet-SA

As described, one of the inductive biases made by EEGNet is that the sequentially ordered sensors are locally related. However, this does not have to be the case. For example, these sensors can be placed far apart on the subject’s head, or perhaps different brain regions cooperate with respect to the task. If this is the case, EEGNet’s spatial convolutions may be sub-optimal, as they do not necessarily equally allow far apart sensors to share information. We therefore decided to replace the spacial convolutions in EEGNet with a Spacial Attention block (SA-block). We call this model EEGNet-SA. The SA-block consists of a simple transformer that uses attention over the sensor channel to predict a CLS token, which can be influenced by all relevant sensors, with different magnitudes. We expect this block to allow the model to better capture the relationships between different spatially located sensors. The rest of the EEGNet architecture can remain in-tact, as the inductive bias holds for the temporal convolution, and the final layers of EEGNet combine the found features, which is still relevant. Our architecture can be found in Figure 1

D. MEG-GPT

Where EEGNet and EEGNet-SA process sensors locally or independently, MEG-GPT [8] attends across all sensors jointly

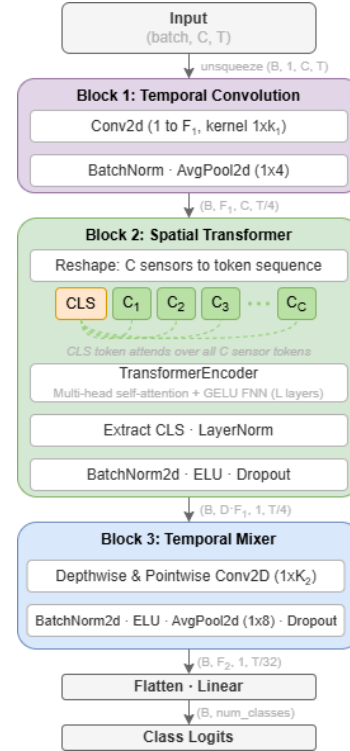


Fig. 1: Architecture of EEGNet-SA. Here, B denotes batch size, C the number of sensors, and T the window size in timesteps.

in a single sequence, introducing a fundamentally different inductive bias from the other models. The original MEG-GPT is a decoder only Transformer pre-trained on MEG data from 612 subjects. This model was selected to test whether pre-training on unlabelled MEG data can outperform architectures trained from scratch.

Since the original model was created using TensorFlow for 38 brain-region time-series data, and our data consists of 248 raw sensor recordings, the core architecture was recreated in PyTorch and adapted to our classification task. The model takes an input tensor of shape $(B \times C \times T)$ in four stages. At the beginning, a patch embedding layer processed each sensor separately. The timeseries of a sensor is passed through a strided Conv1d with kernel size equal to the window size T , resulting in exactly one token per sensor. The tokens from all sensors (C) are concatenated into a single sequence. Then, the sinusoidal positional encodings are added to each token. Furthermore, the sequence is passed through L causal Transformer layers. Causality is consistent with the autoregressive inductive bias of the original MEG-GPT model. Lastly, the token embeddings are mean-pooled into a single vector, then passed through a LayerNorm followed by a linear classification head, resulting in four class logits.

a) *Fine-tuning Strategy:* To make use of the original MEG-GPT model from the publicly available checkpoint, hosted on HuggingFace, we initialise the Transformer core of the model. Two model components are intentionally excluded from the weight transfer: the patch embedding layer, because

its input dimensionality is incompatible with the pre-trained tokeniser, and the classification head. After the pre-trained weights are loaded, the Transformer backbone is frozen, leaving only the patch embedding and the classification head trainable. This approach substantially reduces the number of parameters updated during training, significantly decreasing training time and VRAM usage, allowing us to include the model in this paper. Moreover, this approach also acts as a strong regulariser given the limited amount of data.

E. MEG-GCNet

Our final model is adapted from WeatherGCNet [9], a spatio-temporal graph convolutional network originally developed for wind speed prediction across geographically distributed weather stations. WeatherGCNet itself is built on ST-GCN [10] and 2s-AGCN [11], two architectures originally designed for skeleton-based action recognition, where the joints of the human body are treated as nodes in a spatial graph. This lineage is arguably a closer conceptual fit for our task than the weather application: like body joints, MEG sensors are spatially distributed nodes whose pairwise relationships are not known in advance and must be inferred from the data.

The core inductive bias this model introduces is one that none of our other models share: rather than assuming sensors are locally related by index (EEGNet) or computing pairwise attention scores between all sensors simultaneously (EEGNet-SA), this model represents sensors as nodes in a graph and *learns* the strength of the connections between them directly from the data, resulting in a fixed connectivity structure that is shared across all inputs. This contrasts with the attention mechanism in EEGNet-SA, where sensor interactions are recomputed dynamically for every input sample. We selected this model because it is plausible that only certain subsets of sensors are functionally coupled for a given cognitive task, and a learned adjacency matrix can discover these dependencies in a data-driven way, without requiring hand-crafted assumptions about sensor topology.

Adapting this model to our classification task required two targeted changes from the original regression formulation. The output layer was replaced by a global average pool over both the time and node dimensions, followed by a linear layer producing four class logits. The cross-entropy loss replaced the original mean absolute error. Additionally, two architectural deviations from the original were introduced to make training feasible with our computational resources. First, the channel width was reduced, giving a progression $1 \rightarrow 8 \rightarrow 16 \rightarrow 32$ rather than the original $1 \rightarrow 16 \rightarrow 32 \rightarrow 64$. Second, temporal average pooling with a factor of 4 is applied between each pair of consecutive ST-blocks, rather than the factor of 2 used in the original, more aggressively reducing the time dimension before each subsequent block. Together these changes substantially reduce the computational cost of the graph convolution operations, which scale with the number of channels and the length of the time dimension, while preserving the overall architecture and its inductive bias.

IV. METHOD

This section describes our methodology. First, we discuss our data preparation approach. Then we describe the hyperparameters present in our data and models and how we optimised their values. After which, we discuss how we performed cross-validation using our different subjects. Afterwards, we discuss our performance metrics and our testing approach. Lastly, we mention our hardware restrictions.

A. Sensor Omission

MEG recordings can contain non-neural signals, referred to as artifacts, arising from sources such as the subject moving, or sensor-level malfunction [12]. As MEG data can be easily corrupted by such artifacts, the data may contain signal distributions that deviate substantially from the majority of sensors. Retaining these sensors risks introducing noise that degrades model performance. To mitigate this, we applied an outlier detection procedure to identify and remove these sensors before model training.

For each MEG recording file, we computed the mean signal amplitude across over time for every sensor. Sensors were flagged as outliers and excluded when their mean amplitude fell outside the interval $[Q1 - k * IQR, Q3 + k * IQR]$, where $Q1$ and $Q3$ represent the first and third quartiles of the mean distribution and $IQR = Q3 - Q1$. The multiplier k was set to 4.5. This value was selected through visual inspection of all the recording files before and after outlier detection. Lower multipliers were found to be too aggressive, removing sensors that did not visually appear to be outliers, while higher multipliers failed to remove sensors that did appear to be outliers. 4.5 struck the best balance, reliably isolated only sensors appearing to be outliers. If an outlier was identified in a single recording it was considered an outlier in all files. In order to be thorough, we re-introduced the omitted sensors for the best performing models and report on the results in Appendix E.

B. Hyperparameters

The choice and optimisation of hyperparameters is crucial to achieving high model performance. We identified and optimised six key hyperparameters during the optimisation process: learning rate, weight decay, dropout, batch size, window size, and downsample factor. Their definition and values are explained below.

Learning rate (the step size taken during gradient descent optimization) was optimised in the range $1e-6$ to $1e-1$. This wide range was selected to balance the search between fine-grained convergence at the lower end and rapid initial learning at the higher end, ensuring the model avoids parameter divergence. **Weight decay** (an L2 penalty applied to the model weights to encourage simpler models) was also optimised in the same range. This range covers everything from negligible regularisation to aggressive weight penalisation, allowing the optimiser to find the ideal trade-off between bias and variance. **Dropout** regularisation (the probability of randomly zeroing out hidden units during a forward pass) was optimised between

0.0 and 0.75. This range spans from no dropout to heavy regularisation, stopping at 0.75 because dropping more neurons typically destroys too much representational capacity for the network to learn effectively. **Batch size** (the number of training samples processed before the model’s internal parameters are updated) was optimised between 8 and 128 samples, except for MEG-GPT and MEG-GCNet where the maximum was set at 64 due to computational restrictions. The lower bound of 8 ensures highly stochastic gradient estimates to help escape local minima, while the upper bound of 128 was chosen as the strict maximum size our hardware’s VRAM could reliably handle for the baseline models. **Window size** (the number of consecutive temporal measurements included in a single input sample) was optimised between 32 and 1024 time samples. This range was selected because 32 samples capture rapid, highly localized temporal dynamics, while 1024 samples encompass roughly half a second of MEG recording, allowing the model to capture slower, sustained cognitive processes. Note that there was no overlap between the windows, to mitigate overfitting. The **downsample factor** (the rate at which the temporal dimension of the raw signal is compressed or subsampled) was optimised between 1 and 6. A factor of 1 retains the full signal resolution, while a factor of 6 acts as an empirical upper limit for dimensionality reduction, beyond which the signal loses the high-frequency information necessary to distinguish the specific cognitive tasks.

Several other hyperparameters were kept fixed throughout the experiments. The optimizer was set to Adam, which adapts the learning rate individually for each parameter and is widely used in deep learning due to its robustness. The total number of training epochs was fixed at 30 epochs, providing sufficient time for model convergence without excessive computational cost. These parameters were fixed based on preliminary experimentation and domain knowledge, ensuring a stable baseline that allowed the optimised hyperparameters to be the primary source of variation during optimisation.

C. Hyperparameter Optimisation

Hyperparameter optimisation (HPO) was used to automatically search for well-performing hyperparameter configurations. According to Bischl et al. [13], HPO can be formulated as a computationally expensive optimisation problem, where the goal is to minimise the error of a model over a predefined hyperparameter search space.

Several HPO approaches exist. Traditional methods, such as grid search and random search, evaluate configurations through exhaustive or stochastic sampling of the search space. While these methods are simple to implement, they become less efficient as the dimensionality of the search space grows. More advanced techniques include evolutionary strategies, racing methods, Bayesian Optimisation, and multi-fidelity approaches such as Hyperband [14].

In their paper, Bischl et al. [13] note that traditional approaches (i.e., grid search) are primarily suitable for optimisation problems with only a very limited number (2-3) of hyperparameters. Beyond this scale, the authors recommend

the use of Bayesian Optimisation up to 10 hyperparameters. As the models developed in this work contained multiple hyperparameters with non-trivial search ranges, bayesian optimisation was selected as the primary optimisation method. Additionally, following the Author’s recommendation for computationally expensive settings, hyperband pruning was combined with bayesian optimisation throughout this work.

Bayesian optimisation fits a model to guide the optimisation by balancing exploration and exploitation of the hyperparameter space (see Appendix B for a detailed description). We used 7 startup trials (initial random hyperparameter configurations before bayesian optimisation begins), as recommended by Falkner et al. [15]. As well as 40 total trials (the total number of hyperparameter configurations evaluated) based on the available computational resources. Hyperband on the other hand, reduces computational cost by pruning poorly performing configurations at checkpoints, while allowing promising configurations to train until the next checkpoint (see Appendix C for a detailed description). We set the minimum epochs to 10, based on preliminary experiments. We used a reduction factor of 3 (the factor of configurations that are pruned at each checkpoint), as recommended by Falkner et al. [15].

D. Cross-Validation

To robustly evaluate model performance and ensure proper generalization to unseen data, we employed k-fold cross-validation. It was applied in two distinct settings. For *intra-subject* evaluation, we use 8-fold cross-validation, separating 1 entire file for each class per fold. This gave the most robust validation estimate. For *cross-subject* evaluation, we use 3-fold cross-validation at subject-level (i.e., a single subject for validation per fold). Preventing subject data leakage is particularly important in MEG classification tasks where subject-specific characteristics, such as individual anatomical differences and neural variability, could bias validation performance.

During each cross-validation fold, the model and optimiser states are reset to their initial configuration, ensuring that each fold begins with identical starting weights. This guarantees that differences in fold performance arise from differences in the training data rather than from initialisation artifacts or accumulated states from previous folds. After completing all folds, the model checkpoint that achieved the lowest mean validation loss across all folds is retained as the final model. This selection criterion ensures that we use the best-performing model according to the validation criterion for testing.

E. Metrics

To assess model performance, we used accuracy as the primary evaluation metric: the proportion of correctly classified samples out of the total number of samples. The cross-entropy loss function was used for training. Both are standard approaches for multi-class classification tasks.

During training, we monitored both training loss and validation loss at each epoch to track learning progress and detect

overfitting. Validation loss served as the criterion for model selection during hyperparameter optimisation.

F. Testing

Once hyperparameter optimisation is complete, each model is retrained from scratch on the full available training data using the best-found hyperparameter configuration. This means training on all the combined set of both the training and validation data. For the test set, a new window was sampled for every possible starting index, as to report the most accurate testing results. Each window is classified independently, and the resulting predictions are aggregated to produce the final test accuracy.

G. Hardware and Computational Constraints

Model training, hyperparameter optimisation, and evaluation procedures were distributed across several personal computers. The most capable workstation available to our team was equipped with an AMD Ryzen 5 7600X processor, 32 GB of system RAM, and an NVIDIA GeForce RTX 4060 GPU (MSI Ventus 2X Black 8G OC) providing 8 GB of Video RAM (VRAM).

Processing high-dimensional, multi-channel MEG time-series data is highly memory-intensive. Even when utilising our best available hardware, the experimental setup was fundamentally bottlenecked by the 8 GB of available VRAM. This limitation directly influenced our experimental design, restricting the upper bounds of spatial and temporal hyperparameters, such as batch size and window size, to prevent out-of-memory errors during training. Consequently, we had to introduce structural modifications to the architectures of our larger models.

V. RESULTS

This section elaborates on our experimental results. First, we discuss the results of hyperparameter optimisation, where we analyse which hyperparameter configurations perform best for each model as well as the training and validation performances. After which we discuss the results of the best performing models on the test data.

A. Optimisation Results

Tables I and II report the best-found hyperparameter combinations per model for the intra-subject and cross-subject data partitions respectively, with the optimisation trajectories visualised in Appendix D. Notably, the optimal hyperparameter values show less variation across models in the intra-subject partition compared to the cross-subject partition.

Table III reports the corresponding training and validation accuracy during hyperparameter optimisation. For the intra-subject partition (Table IIIa), the majority of models achieve (near) perfect training and validation accuracy. These results are in line with prior research, who find that capturing the pattern of a single subject is near trivial [6]. Both EEGNet-SA and MEG-GPT fail to reach perfect validation accuracy. Their high standard deviations (17.26 and 14.42, respectively) suggest they are sensitive to fold composition, implying a tendency to overfit to specific folds rather than generalizing across different splits of the same subject's data.

The cross-subject partition (Table IIIb) is more challenging as models are evaluated on a subject unseen during training, resulting decrease in validation accuracy across all models. EEGNet achieves the highest validation performance of $74.04 \pm 12.05\%$, though again the high standard deviation suggests its performance is sensitive to which subjects are in the train set. Figure 2 shows the training and validation loss curves for this model, where the clear divergence between the two indicates overfitting, despite achieving the best validation performance among all models. This divergence is not unique to EEGNet. Unlike the intra-subject partition, training accuracy is no longer similar to validation performance. The Baseline achieves $100.0 \pm 0.00\%$ training accuracy, but only $61.06 \pm 10.26\%$ validation accuracy, which is a clear sign of overfitting. EEGNet-SA performs the worst in validation ($39.40 \pm 5.93\%$), despite reasonable training accuracy ($81.11 \pm 2.89\%$). Table IV shows the confusion matrix of this model, here we can observe this model predicting **rest** most of the time and to a lesser extent **working memory**, suggesting the patterns learned from the train subjects do not transfer well to unseen subjects.

TABLE I: Best found hyperparameter combinations per model for the **intra-subject** data partition.

	Learning Rate	Weight decay	Dropout	Batch size	Window size	Downsample factor
Baseline	$3.14\text{e-}4$	$4.34\text{e-}6$	0.136	30	334	4
EEGNet	$6.90\text{e-}4$	$1.60\text{e-}5$	0.410	10	629	1
EEGNet-SA	$1.55\text{e-}4$	$1.24\text{e-}6$	0.295	8	234	3
MEG-GPT*	$6.42\text{e-}5$	$1.04\text{e-}6$	0.024	49	353	1
MEG-GCNet*	$8.75\text{e-}4$	$1.54\text{e-}5$	0.015	8	362	1

*Maximum for batch size was 64 instead of 128.

TABLE II: Best found hyperparameter combinations per model for the **cross-subject** data partition.

	Learning Rate	Weight decay	Dropout	Batch size	Window size	Downsample factor
Baseline	$8.19\text{e-}6$	$8.82\text{e-}5$	0.122	112	74	5
EEGNet	$2.26\text{e-}4$	$6.63\text{e-}6$	0.036	11	253	6
EEGNet-SA	$2.87\text{e-}5$	$1.11\text{e-}4$	0.007	52	205	2
MEG-GPT*	$5.10\text{e-}6$	$2.76\text{e-}6$	0.023	30	1012	1
MEG-GCNet*	$4.34\text{e-}4$	$2.98\text{e-}5$	0.472	27	461	4

*Maximum for batch size was 64 instead of 128.

TABLE III: Accuracy (%) of the best found hyperparameters configurations per model, as averaged over the folds, including standard deviation. The best performance is highlighted.

(a) Intra-subject

Model	Train	Val.
Baseline	95.65 $\pm$ 0.53	100.0 $\pm$ 0.00
EEGNet	96.63 $\pm$ 0.46	100.0 $\pm$ 0.00
EEGNet-SA	90.91 $\pm$ 1.34	88.44 $\pm$ 17.26
MEG-GPT	90.43 $\pm$ 1.01	89.37 $\pm$ 14.42
MEG-GCNet	98.07 $\pm$ 0.64	100.0 $\pm$ 0.00

(b) Cross-subject

Model	Train	Val.
Baseline	100.0 $\pm$ 0.00	61.06 $\pm$ 10.26
EEGNet	99.93 $\pm$ 0.06	74.04 $\pm$ 12.05
EEGNet-SA	81.11 $\pm$ 2.89	39.40 $\pm$ 5.93
MEG-GPT	87.80 $\pm$ 2.27	52.11 $\pm$ 7.09
MEG-GCNet	75.20 $\pm$ 1.96	62.29 $\pm$ 9.42

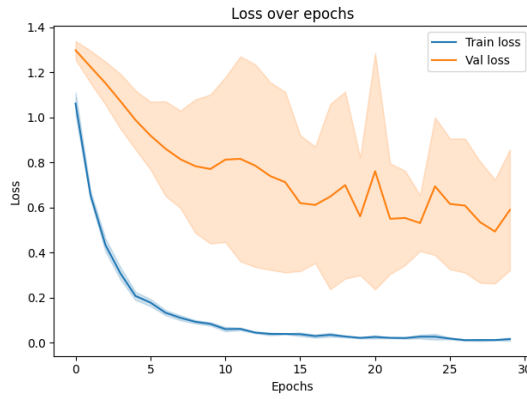


Fig. 2: Loss graph during training of EEGNet with the best found hyperparameters on the cross-subject data partition.

TABLE IV: Normalised Confusion Matrix of the predictions for EEGNet-SA on the cross-subject partition.

		Predicted			
		rest	motor	story & math	working memory
Actual	rest	0.67	0.05	0.05	0.23
	motor	0.66	0.04	0.06	0.24
	story & math	0.44	0.10	0.09	0.38
	working memory	0.58	0.01	0.05	0.37

B. Test Results

Table V reports train and test accuracy of all models for both data partitions. In the intra-subject setting, results largely confirm what was observed during validation: Baseline, EEGNet, MEG-GPT and MEG-GCNet all achieve (near) 100.0% train and test accuracy. However, EEGNet-SA achieves a slightly lower test accuracy (83.41%) compared to its validation accuracy (90.91 $\pm$ 1.34%).

TABLE V: Accuracy (%) for best found configurations per model. The best performance is highlighted.

Model	Intra-subject		Cross-subject	
	Train	Test	Train	Test
Baseline	100.0	100.0	100.0	62.34
EEGNet	100.0	100.0	100.0	64.20
EEGNet-SA	95.75	83.41	42.30	29.33
MEG-GPT	94.75	97.08	69.91	49.31
MEG-GCNet	100.0	99.58	99.95	49.85

In the cross-subject setting, EEGNet again achieves the best test accuracy (64.20%), marginally outperforming the Baseline (62.34%). The remaining models perform considerably worse, for MEG-GCNet (49.85%) and MEG-GPT (49.31%), inspection of their predictions reveals that both predominantly predict **rest** and **motor**, limiting their effective accuracy to roughly 50%. EEGNet-SA performs the worst by far (29.33%), consistent with its poor performance during hyperparameter optimisation.

VI. CONCLUSION

This study investigated to what capacity modern AI models are able to capture patterns in MEG data in order to achieve high classification accuracy under highly data constrained conditions, both within and across multiple subjects. We trained five progressively more complex models to a small MEG dataset and were able to achieve perfect performance on the intra-subject partition with both our baseline and EEGNet. For the cross-subject partition, EEGNet was our best performing model with 64.20% accuracy on the test partition. Our results suggest that fitting a model within a single subject is not that challenging, though more complex models seem to struggle slightly more than simple models. Regarding the cross-subject partition, we find all models struggle. Our best performing model outperformed the baseline by only 1.86 percentage points. We conclude that even state of the art models struggle to generalise on MEG data, under highly cross-subject data constrained conditions.

For future research we recommend considering approaches to enrich data when in similar constrained environments. For example, data augmentation techniques to enrich data and prevent overfitting, or pre-training on similar domains or data to encourage the model to capture relevant patterns rather than subject-specific attributes.

REFERENCES

- [1] H. Berger, "Über das elektroencephalogramm des menschen," *Archiv für Psychiatrie und Nervenkrankheiten*, vol. 87, no. 1, pp. 527–570, Dec 1929. [Online]. Available: <https://doi.org/10.1007/BF01797193>
- [2] A. Craik, Y. He, and J. L. Contreras-Vidal, "Deep learning for electroencephalogram (EEG) classification tasks: a review," *J Neural Eng*, vol. 16, no. 3, p. 031001, Feb. 2019.
- [3] D. M. Praveena, D. A. Sarah, and S. T. George, "Deep learning techniques for eeg signal applications – a review," *IETE Journal of Research*, vol. 68, no. 4, pp. 3030–3037, 2022. [Online]. Available: <https://doi.org/10.1080/03772063.2020.1749143>

- [4] O. Presacan, J. Ojha, A. Yazidi, E. Monteiro, and P. G. Lind, "A comprehensive review of explainable ai in deep learning algorithms for eeg analysis," *ACM Trans. Comput. Healthcare*, vol. 7, no. 2, Mar. 2026. [Online]. Available: <https://doi.org/10.1145/3785409>
- [5] D. Cohen, "Magnetoencephalography: Evidence of magnetic fields produced by alpha-rhythm currents," *Science*, vol. 161, no. 3843, pp. 784–786, 1968. [Online]. Available: <https://www.science.org/doi/abs/10.1126/science.161.3843.784>
- [6] A. Dehgan, H. Abdelhedi, V. Hadid, I. Rish, and K. Jerbi, "Artificial neural networks for magnetoencephalography: a review of an emerging field," *Journal of Neural Engineering*, vol. 22, no. 3, p. 031001, jun 2025. [Online]. Available: <https://doi.org/10.1088/1741-2552/add4a>
- [7] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung, and B. J. Lance, "Eegnet: a compact convolutional neural network for eeg-based brain–computer interfaces," *Journal of Neural Engineering*, vol. 15, no. 5, p. 056013, 2018. [Online]. Available: <http://dx.doi.org/10.1088/1741-2552/aace8c>
- [8] R. Huang, S. Cho, C. Gohil, O. P. Jones, and M. Woolrich, "Meg-gpt: A transformer-based foundation model for magnetoencephalography data," 2025. [Online]. Available: <https://arxiv.org/abs/2510.18080>
- [9] T. Stańczyk and S. Mehrkanoon, "Deep graph convolutional networks for wind speed prediction," 2021. [Online]. Available: <https://arxiv.org/abs/2101.10041>
- [10] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," 2018. [Online]. Available: <https://arxiv.org/abs/1801.07455>
- [11] L. Shi, Y. Zhang, J. Cheng, and H. Lu, "Two-stream adaptive graph convolutional networks for skeleton-based action recognition," 2019. [Online]. Available: <https://arxiv.org/abs/1805.07694>
- [12] M. Jas, D. A. Engemann, Y. Bekhti, F. Raimondo, and A. Gramfort, "Autorejct: Automated artifact rejection for meg and eeg data," *NeuroImage*, vol. 159, pp. 417–429, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1053811917305013>
- [13] B. Bischl, M. Binder, M. Lang, T. Pielok, J. Richter, S. Coors, J. Thomas, T. Ullmann, M. Becker, A.-L. Boulesteix, D. Deng, and M. Lindauer, "Hyperparameter optimization: Foundations, algorithms, best practices and open challenges," 2021. [Online]. Available: <https://arxiv.org/abs/2107.05847>
- [14] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," 2018. [Online]. Available: <https://arxiv.org/abs/1603.06560>
- [15] S. Falkner, A. Klein, and F. Hutter, "Bohb: Robust and efficient hyperparameter optimization at scale," 2018. [Online]. Available: <https://arxiv.org/abs/1807.01774>

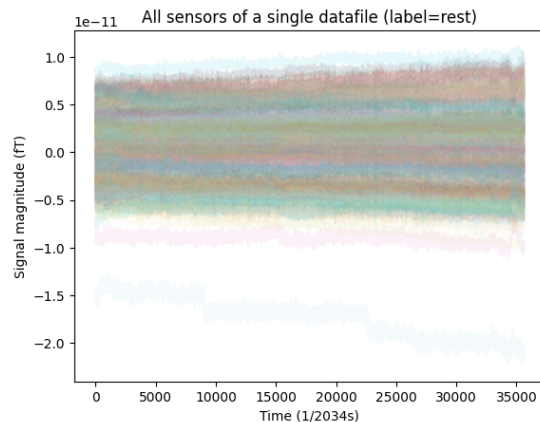


Fig. 3: Line charts showing a unique line for each sensor of a resting-state datafile from the intra-subject partition.

APPENDIX A DATA VISUALISATION

Figures 3 shows an example of one single training file, from the resting label of the subject in the intra-subject data partition. It can be seen from this figure that some of the sensors are clear outliers.

APPENDIX B BAYESIAN OPTIMISATION

Bayesian optimisation iteratively fits a surrogate model (typically a gaussian process) that approximates the relationship between any given hyperparameter configuration and model performance (e.g. validation loss). The optimisation process starts with a number of startup trials consisting of randomly sampled hyperparameter configurations, which provide initial observations for fitting the surrogate. After these evaluations, the surrogate is updated after each subsequent trial. An acquisition function then uses the surrogate’s predicted performance and associated uncertainty to score candidate configurations, balancing exploration of unexplored configurations against exploitation of promising ones. The configuration with the highest acquisition score is selected for evaluation next, and the cycle repeats until a pre determined number of configurations is reached.

APPENDIX C HYPERBAND PRUNING

Instead of fully training every hyperparameter configuration for the maximum number of epochs, Hyperband initially evaluates configurations using a limited computational budget. At predefined checkpoints, occurring at multiples of the predefined minimum number of epochs, the performance of a configuration is compared against past trials. Configurations that perform poorly at these checkpoints are pruned early, while promising configurations are allowed to continue training with an increased computational budget. The number of trials pruned at each checkpoint is controlled through a predefined reduction factor.

APPENDIX D HYPERPARAMETER OPTIMISATION RESULTS

Figures 4-8 show parallel coordinate plots for all five models trained on the intra-subject data partition (baseline, EEGNet, EEGNet-SA, MEG-GPT, and MEG-GCNet, respectively). Figures 9-13 show corresponding plots for the cross-subject data partition, in the same model order. Each line in the plots represents a single hyperparameter configuration evaluated during hyperparameter optimisation, coloured by validation loss (yellow = low, blue = high). The plots allow visual inspection of which hyperparameter values were explored and which configurations tended to yield lower or higher validation loss. It is worth noting that trials that were pruned early do not appear in these figures.

APPENDIX E

EFFECT OF SENSOR OMISSION

While sensor omission was applied to reduce the influence of artifactual signals, it also removes potentially relevant features from the dataset. To assess whether this trade-off is beneficial, Table VI reports train and test accuracy of all models when all sensors are included. In the intra-subject partition, performance remains largely unchanged, with a big drop in train and test accuracy for EEGNet-SA. In the cross-subject partition, all models improve their test performance, with the largest gains seen in EEGNet-SA (+14.46) and MEG-GCNet (+12.48), suggesting that these models are better able to exploit the additional spatial information provided by the full sensor set.

TABLE VI: Accuracy (%) for best model configurations without omitted sensors. Best performance is highlighted.

Model	Intra-subject		Cross-subject	
	Train	Test	Train	Test
Baseline	100.0	100.0	100.0	64.49
EEGNet	100.0	100.0	100.0	68.39
EEGNet-SA	53.94	70.15	81.80	43.79
MEG-GPT	95.75	95.57	88.81	52.90
MEG-GCNet	100.0	100.0	100.0	62.33

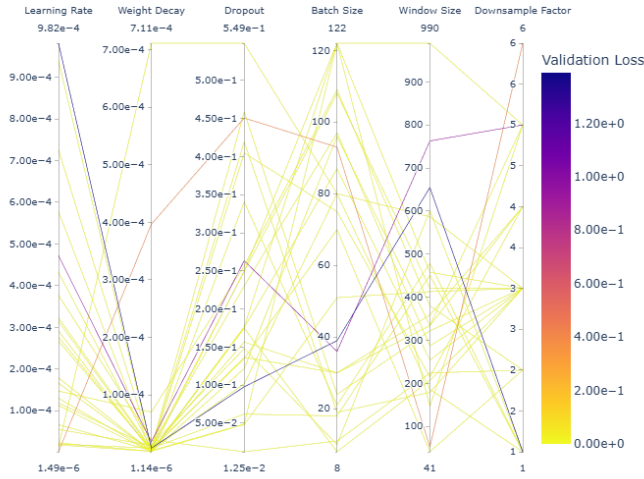


Fig. 4: Hyperparameter optimisation results for the **baseline** model on the **intra-subject** data partition.

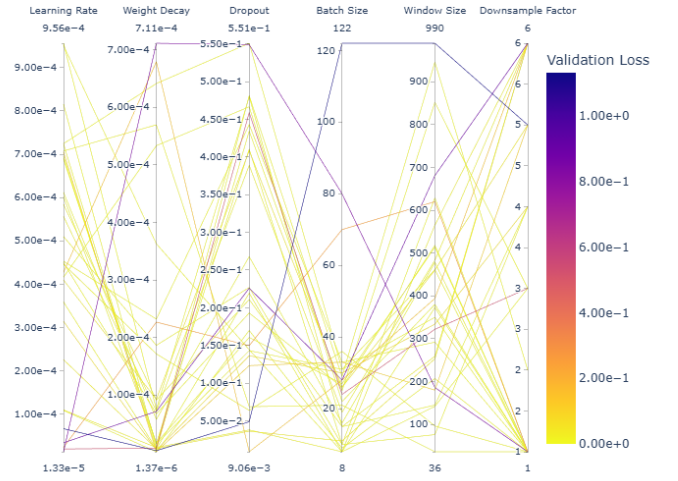


Fig. 5: Hyperparameter optimisation results for the **EEGNet** model on the **intra-subject** data partition.

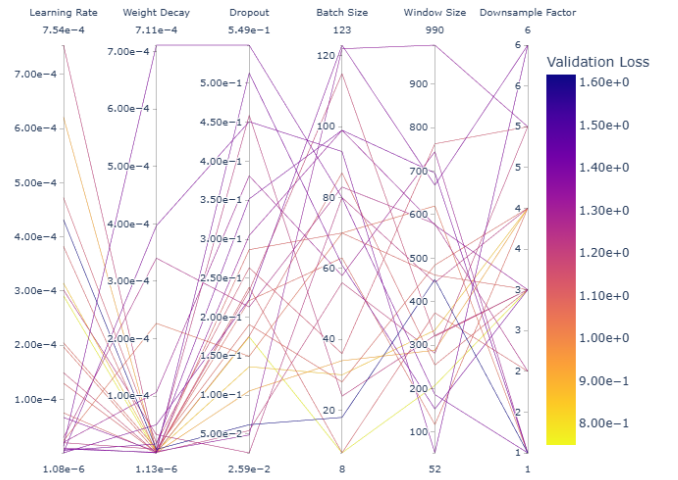


Fig. 6: Hyperparameter optimisation results for the **EEGNet-SA** model on the **intra-subject** data partition.

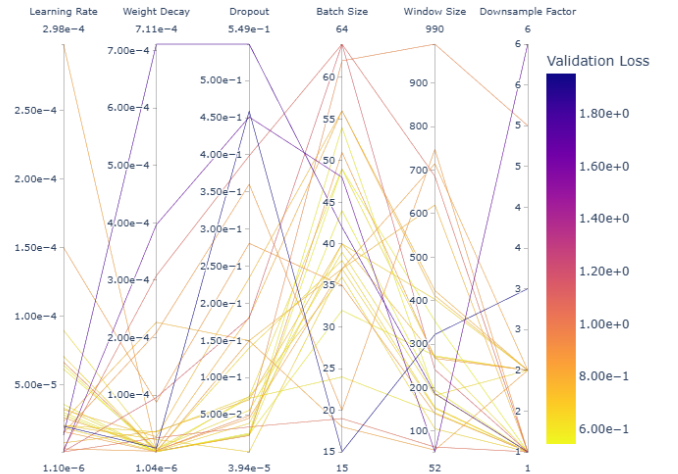


Fig. 7: Hyperparameter optimisation results for the **MEG-GPT** model on the **intra-subject** data partition.

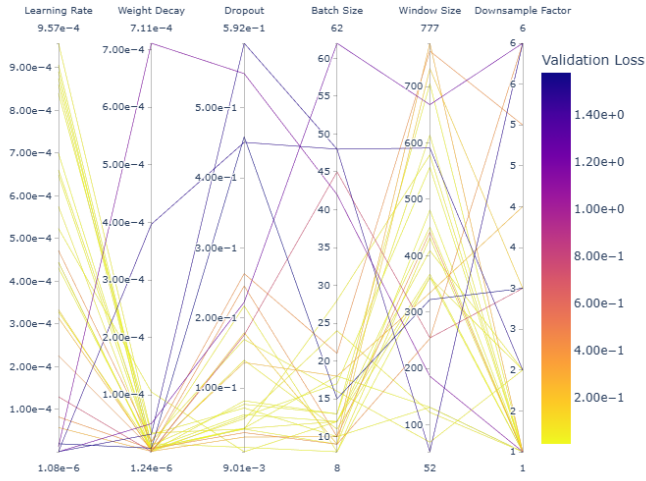


Fig. 8: Hyperparameter optimisation results for the **MEG-GCNet** model on the **intra-subject** data partition.

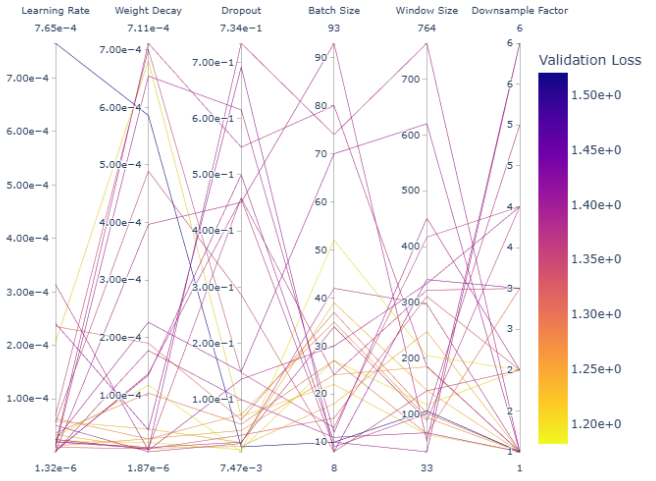


Fig. 11: Hyperparameter optimisation results for the **EEGNet-SA** model on the **cross-subject** data partition.

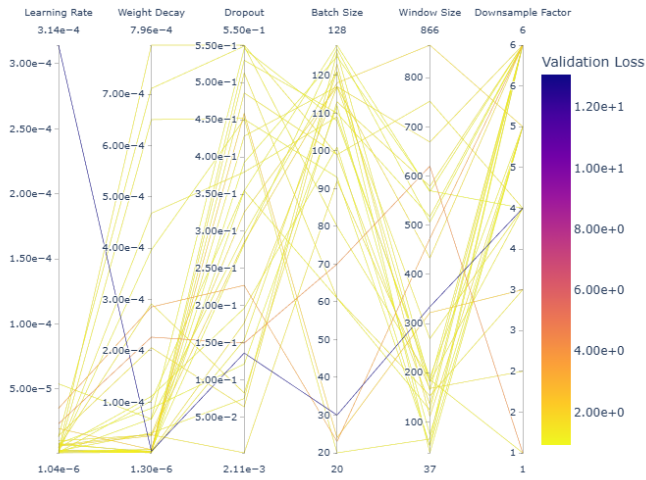


Fig. 9: Hyperparameter optimisation results for the **baseline** model on the **cross-subject** data partition.

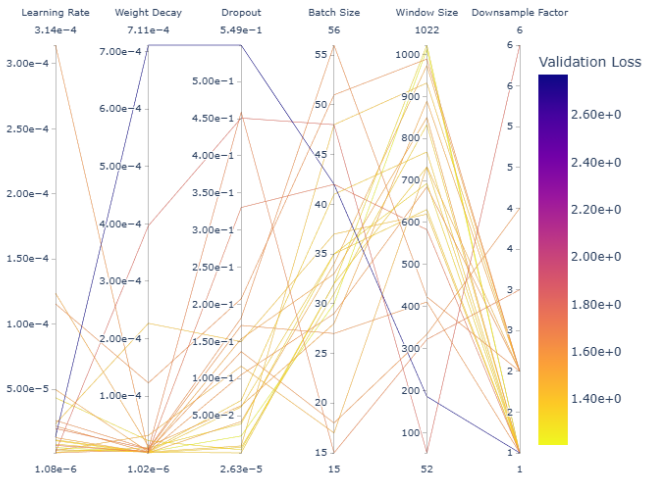


Fig. 12: Hyperparameter optimisation results for the **MEG-GPT** model on the **cross-subject** data partition.

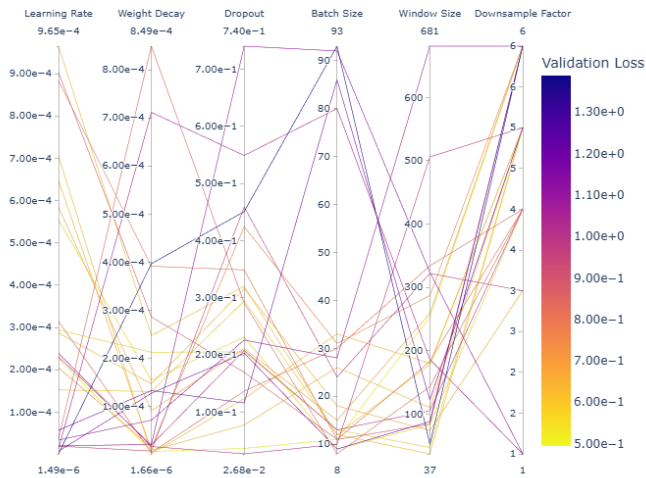


Fig. 10: Hyperparameter optimisation results for the **EEGNet** model on the **cross-subject** data partition.

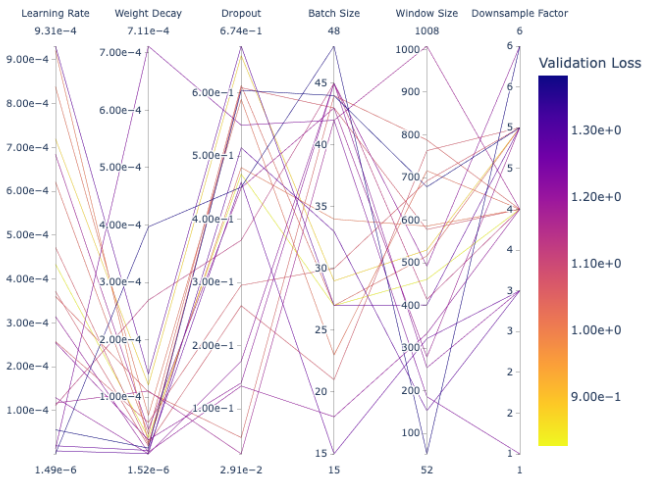


Fig. 13: Hyperparameter optimisation results for the **MEG-GCNet** model on the **cross-subject** data partition.